Лабораторная работа № 2

Структуры данных

Пакавира Арсениу Висенте Луиш

Содержание

1 Цель работы **4**

2 Задание **5**

3 Теоретическое введение **6**

4 Выполнение лабораторной работы **7**

5 Выводы **18**

Список литературы **19**

Список иллюстраций

4.1 Примеры использования кортежей 7
4.2 Примеры использования кортежей 8
4.3 Примеры использования словарей 8
4.4 Примеры использования словарей 8
4.5 Примеры использования множеств 9
4.6 Примеры использования множеств 9
4.7 Примеры использования массивов 10
4.8 Примеры использования массивов 11
4.9 Примеры использования массивов 12
4.10 Задание №1. Работа с множествами 13
4.11 Задание №2. Примеры операций над множествами элементов разных типов 13
4.12 Задание №3. Работа с массивами 13
4.13 Задание №3. Работа с массивами 14
4.14 Задание №3. Работа с массивами 14
4.15 Задание №3. Работа с массивами 14
4.16 Задание №3. Работа с векторами 15
4.17 Задание №3. Работа с векторами 15
4.18 Задание №3. Работа с векторами 15
4.19 Задание №3. Работа с векторами 15
4.20 Задание №3. Работа с векторами 15
4.21 Задание №3. Работа с векторами 16
4.22 Задание №3. Работа с векторами 16
4.23 Задание №4 16
4.24 Задание №5. Работа с пакетом Primes 17
4.25 Задание №6 17

1 Цель работы

Основная цель работы – изучить несколько структур данных, реализованных в Julia, научиться применять их и операции над ними для решения задач.

2 Задание

1. Используя Jupyter Lab, повторите примеры.
2. Выполните задания для самостоятельной работы.

3 Теоретическое введение

Julia – высокоуровневый свободный язык программирования с динамической типизацией, созданный для математических вычислений [1]. Эффективен также и для написания программ общего назначения. Синтаксис языка схож с синтаксисом других математических языков, однако имеет некоторые существенные отличия.

Для выполнения заданий была использована официальная документация Julia [2].

Рассмотрим несколько структур данных, реализованных в Julia. Несколько функций (методов), общих для всех структур данных:

- isempty() – проверяет, пуста ли структура данных;
- length() – возвращает длину структуры данных;
- in() – проверяет принадлежность элемента к структуре;
- unique() – возвращает коллекцию уникальных элементов структуры,
- reduce() – свёртывает структуру данных в соответствии с заданным бинарным оператором;
- maximum() (или minimum()) – возвращает наибольший (или наименьший) результат вызова функции для каждого элемента структуры данных.

4 Выполнение лабораторной работы

Для начала выполним примеры из раздела про кортежи (рис. 4.1-4.2).

Кортеж (Tuple) – структура данных (контейнер) в виде неизменяемой индексируемой последовательности элементов какого-либо типа (элементы индексируются с единицы).



Рис. 4.1: Примеры использования кортежей

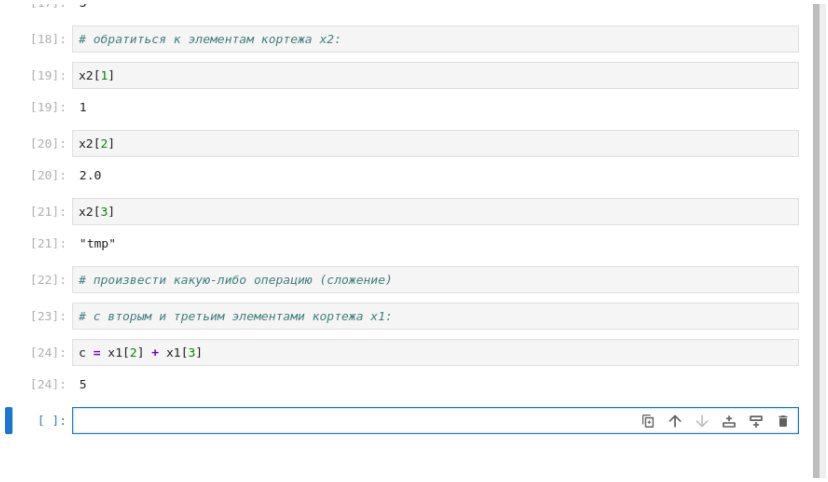


Рис. 4.2: Примеры использования кортежей

Теперь выполним примеры из раздела про словари (рис. 4.3-**??**).

Словарь – неупорядоченный набор связанных между собой по ключу данных.

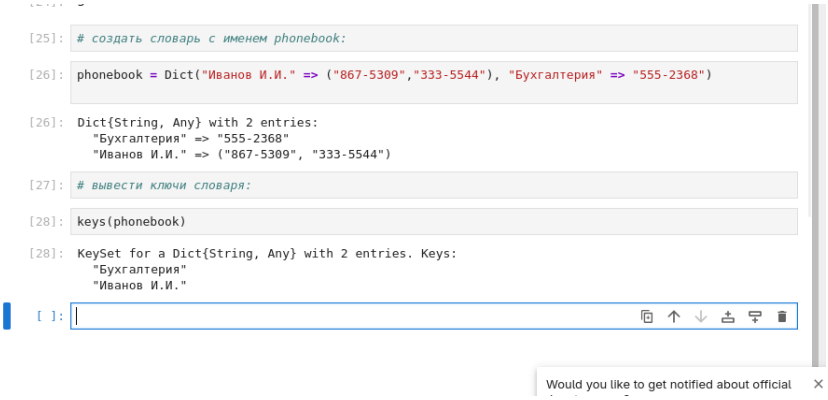


Рис. 4.3: Примеры использования словарей

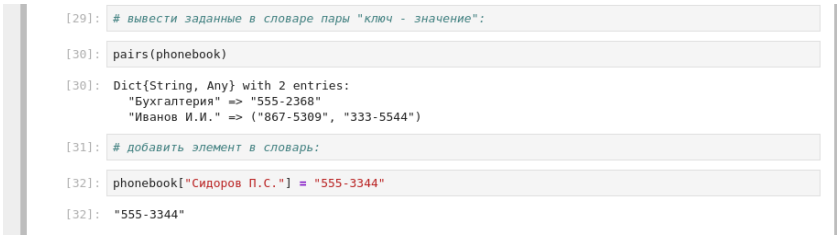


Рис. 4.4: Примеры использования словарей

Выполним примеры из раздела про множества (рис. 4.5-4.6).

Множество, как структура данных в Julia, соответствует множеству, как математическому объекту, то есть является неупорядоченной совокупностью элементов какого-либо типа. Возможные операции над множествами: объединение, пересечение, разность; принадлежность элемента множеству.

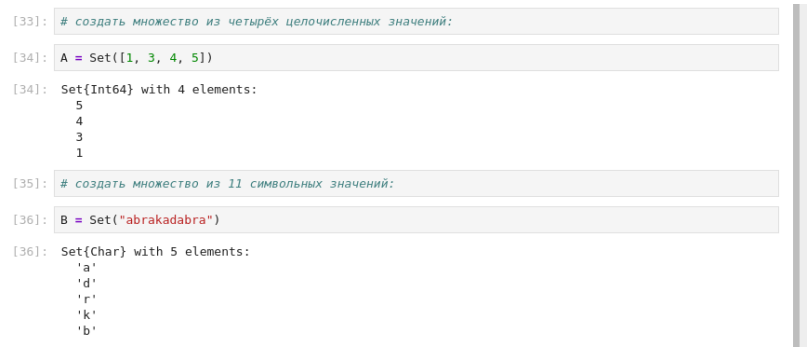


Рис. 4.5: Примеры использования множеств

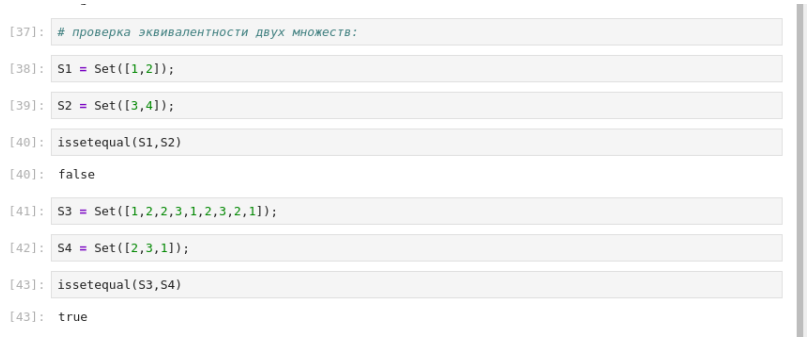


Рис. 4.6: Примеры использования множеств

Выполним примеры из раздела про массивы (рис. 4.5-4.9).

Массив — коллекция упорядоченных элементов, размещённая в многомерной сетке. Векторы и матрицы являются частными случаями массивов.

```
In [112…  # создание пустого массива с абстрактным типом:
          empty_array_1 = []

Out[112…  Any[]

In [114…  # создание пустого массива с конкретным типом:
          empty_array_2 = (Int64)[]
          empty_array_3 = (Float64)[]

Out[114…  Float64[]

In [116…  # вектор-столбец:
          a = [1, 2, 3]

Out[116…  3-element Vector{Int64}:
           1
           2
           3

In [119…  # вектор-строка:
          b = [1 2 3]

Out[119…  1×3 Matrix{Int64}:
           1  2  3

In [121…  # многомерные массивы (матрицы):
          A = [[1, 2, 3] [4, 5, 6] [7, 8, 9]]
          B = [[1 2 3]; [4 5 6]; [7 8 9]]

Out[121…  3×3 Matrix{Int64}:
           1  2  3
           4  5  6
           7  8  9

In [123…  A

Out[123…  3×3 Matrix{Int64}:
           1  4  7
           2  5  8
           3  6  9

In [125…  B

Out[125…  3×3 Matrix{Int64}:
           1  2  3
           4  5  6
           7  8  9

In [144…  # одномерный массив из 8 элементов (массив $1 \times 8$)
          # со значениями, случайно распределёнными на интервале [0, 1):
          c = rand(1,8)

Out[144…  1×8 Matrix{Float64}:
           0.796904  0.960987  0.465085  0.69098  …  0.906977  0.775043  0.465763

In [146…  # многомерный массив $2 \times 3$ (2 строки, 3 столбца) элементов
          # со значениями, случайно распределёнными на интервале [0, 1):
          C = rand(2,3);

In [148…  C
```

Рис. 4.7: Примеры использования массивов

```
In [150…  # трёхмерный массив:
          D = rand(4, 3, 2)

Out[150…  4×3×2 Array{Float64, 3}:
          [:, :, 1] =
           0.713405  0.432203  0.484381
           0.109381  0.536658  0.921419
           0.186919  0.730975  0.475927
           0.216667  0.261629  0.63092

          [:, :, 2] =
           0.0173873  0.26553   0.181904
           0.5598     0.439747  0.789622
           0.276212   0.536542  0.465716
           0.104413   0.577015  0.31033

In [154…  # массив из квадратных корней всех целых чисел от 1 до 10:
          roots = [sqrt(i) for i in 1:10]

Out[154…  10-element Vector{Float64}:
           1.0
           1.4142135623730951
           1.7320508075688772
           2.0
           2.23606797749979
           2.449489742783178
           2.6457513110645907
           2.8284271247461903
           3.0
           3.1622776601683795

In [156…  # массив с элементами вида 3*x^2,
          # где x - нечётное число от 1 до 9 (включительно)
          ar_1 = [3*i^2 for i in 1:2:9]

Out[156…  5-element Vector{Int64}:
            3
           27
           75
          147
          243

In [160…  # массив квадратов элементов, если квадрат не делится на 5 или 4:
          ar_2=[i^2 for i=1:10 if (i^2%5!=0 && i^2%4!=0)]

Out[160…  4-element Vector{Int64}:
            1
            9
           49
           81

In [167…  # одномерный массив из пяти единиц:
          ones(5)

Out[167…  5-element Vector{Float64}:
           1.0
           1.0
           1.0
           1.0
           1.0
```

Рис. 4.8: Примеры использования массивов

```
In [171…  # одномерный массив из 4 нулей:
          zeros(4)

Out[171…  4-element Vector{Float64}:
           0.0
           0.0
           0.0
           0.0

In [173…  # заполнить массив 3x2 цифрами 3.5
          fill(3.5,(3,2))

Out[173…  3×2 Matrix{Float64}:
           3.5  3.5
           3.5  3.5
           3.5  3.5

In [183…  # заполнение массива посредством функции repeat():
          repeat([1,2],3,3)
          repeat([1 2],3,3)

Out[183…  3×6 Matrix{Int64}:
           1  2  1  2  1  2
           1  2  1  2  1  2
           1  2  1  2  1  2

In [185…  # преобразование одномерного массива из целых чисел от 1 до 12
          # в двумерный массив 2x6
          a = collect(1:12)
          b = reshape(a,(2,6))

Out[185…  2×6 Matrix{Int64}:
           1  3  5  7   9  11
           2  4  6  8  10  12

In [187…  # транспонирование
          b'

Out[187…  6×2 adjoint(::Matrix{Int64}) with eltype Int64:
            1   2
            3   4
            5   6
            7   8
            9  10
           11  12

In [189…  # транспонирование
          c = transpose(b)

Out[189…  6×2 transpose(::Matrix{Int64}) with eltype Int64:
            1   2
            3   4
            5   6
            7   8
            9  10
           11  12

In [194…  # массив 10x5 целых чисел в диапазоне [10, 20]:
          ar = rand(10:20, 10, 5)

Out[194…  10×5 Matrix{Int64}:
           18  11  15  12  19
           18  12  20  19  18
           16  14  19  11  15
           12  12  14  18  17
```

Рис. 4.9: Примеры использования массивов

Перейдем к выполнению заданий.

1. Даны множества: $A = 0, 3, 4, 9, B = 1, 3, 4, 7, C = 0, 1, 2, 4, 7, 8, 9$. Найдем $P = A \cap B \cup A \cap B \cup A \cap C \cup B \cap C$ (рис. 4.10).

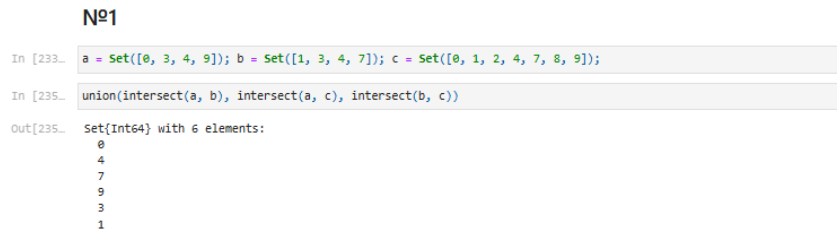


Рис. 4.10: Задание №1. Работа с множествами

2. Приведем свои примеры с выполнением операций над множествами элементов разных типов (рис. 4.11).



Рис. 4.11: Задание №2. Примеры операций над множествами элементов разных типов

3. Создадим массивы разными способами, используя циклы (рис. 4.12-4.22):

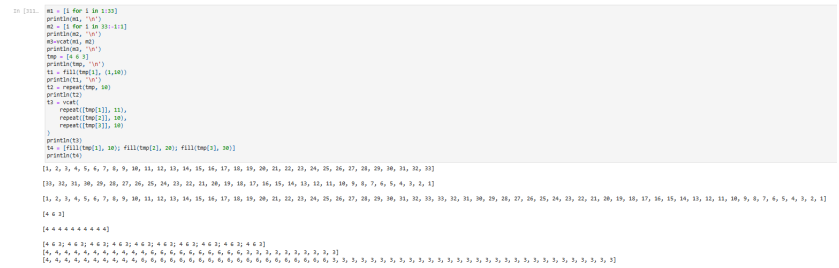


Рис. 4.12: Задание №3. Работа с массивами

[102]:
```julia
# 3.9
tmp = [4, 6, 3]                    # ← garante que tmp existe

tt = []
for i in 1:3
    push!(tt, 2^tmp[i])
end

a39 = vcat(tt, repeat([tt[3]], 3))
println("a39 = ", a39)

k = 0
for i in a39
    if '6' in string(i)
        k += 1
    end
end
println("Цифра 6 встречается ", k, " раз")
```

```
a39 = Any[16, 64, 8, 8, 8, 8]
Цифра 6 встречается 2 раз
```

[ ]:

Рис. 4.13: Задание №3. Работа с массивами

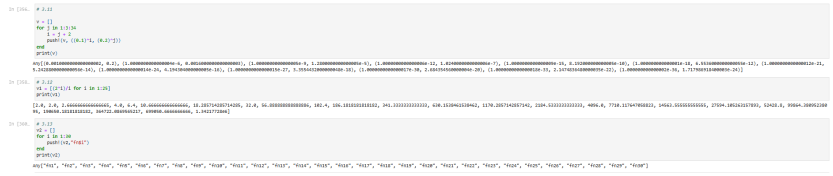


Рис. 4.14: Задание №3. Работа с массивами

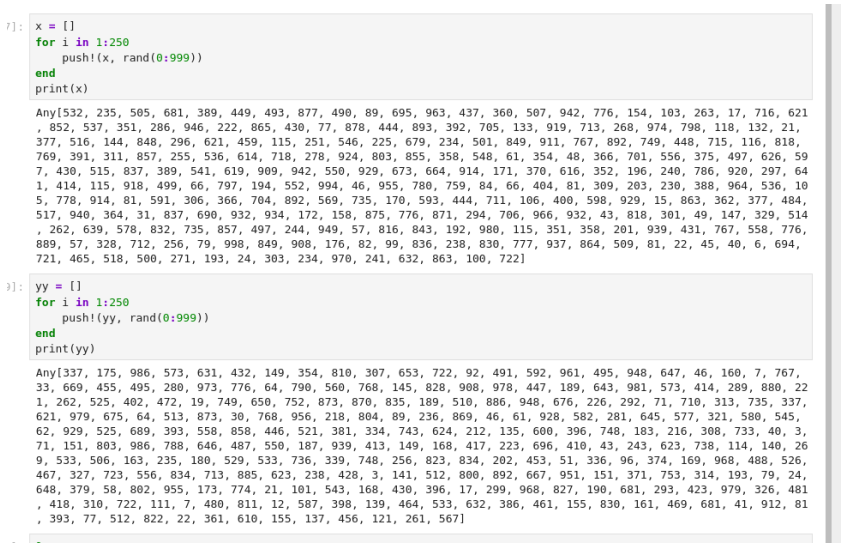


Рис. 4.15: Задание №3. Работа с массивами

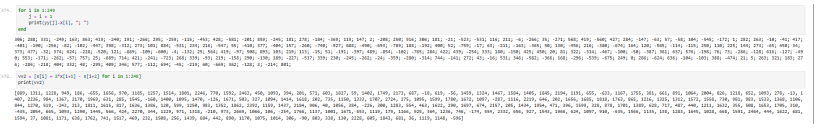


Рис. 4.16: Задание №3. Работа с векторами



Рис. 4.17: Задание №3. Работа с векторами

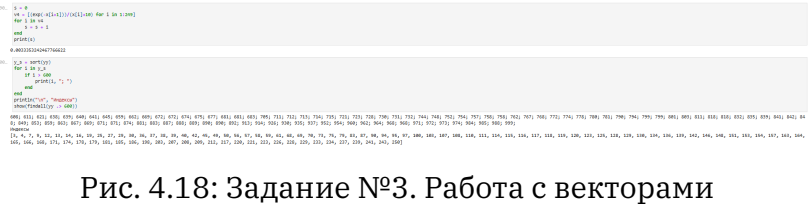


Рис. 4.18: Задание №3. Работа с векторами



Рис. 4.19: Задание №3. Работа с векторами

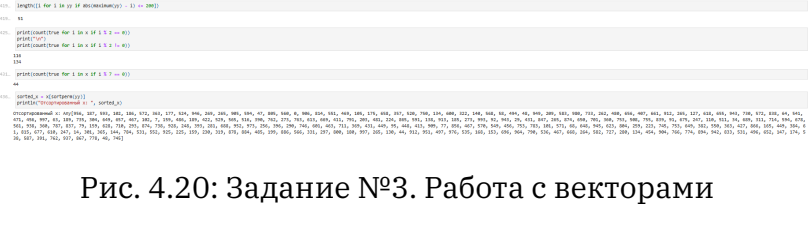


Рис. 4.20: Задание №3. Работа с векторами

```
In [432…  reverse(last(sort(x), 10))

Out[432…  10-element Vector{Any}:
           997
           997
           993
           980
           976
           973
           964
           956
           952
           951
```

Рис. 4.21: Задание №3. Работа с векторами

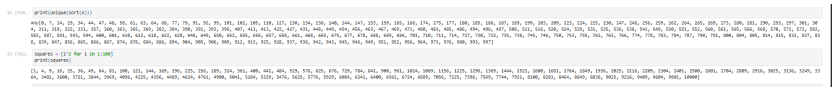


Рис. 4.22: Задание №3. Работа с векторами

4. Создадим массив squares, в котором будут храниться квадраты всех целых чисел от 1 до 100 (рис. 4.23).

```
In [456…  using Primes
          myprime = primes(prime(168))

Out[456…  168-element Vector{Int64}:
             2
             3
             5
             7
            11
            13
            17
            19
            23
            29
            31
            37
            41
             ⋮
           919
           929
           937
           941
           947
           953
           967
           971
           977
           983
           991
           997
```

Рис. 4.23: Задание №4

5. Подключим пакет Primes (функции для вычисления простых чисел). Сгенерируем массив myprimes, в котором будут храниться первые 168 простых чисел. Определим 89-е наименьшее простое число. Получии срез массива с 89-го до 99-го элемента включительно, содержащий наименьшие простые числа (рис. 4.24).

```
In [459…  prime(89)

Out[459…  461

In [461…  myprime1 = [prime(i) for i in 89:99]

Out[461…  11-element Vector{Int64}:
           461
           463
           467
           479
           487
           491
           499
           503
           509
           521
           523
```

Рис. 4.24: Задание №5. Работа с пакетом Primes

6. Вычислим следующие выражения (рис. 4.25).

```
In [473…  s = 0
          for i in 10:100
              s = s + (i^3 + 4*i^2)
          end
          print(s)

          26852735

In [475…  s = 0
          for i in 1:25
              s = s + ((2^i)/(i) + (3^i)/(i^2))
          end
          print(s)

          2.1291704368143802e9

In [477…  s = 1
          a1 = 1
          for i in 2:2:38
              a1 = a1 * i/(i+1)
              s = s + a1
          end
          print(s)

          6.97634613789762
```

Рис. 4.25: Задание №6

5 Выводы

В результате выполнения данной лабораторной работы я изучил несколько структур данных, реализованных в Julia, научилась применять их и операции над ними для решения задач.

Список литературы

1. The Julia Programming Language [Электронный ресурс]. URL: https://julialang.org/ (дата обращения: 11.09.2026).
2. Julia Documentation [Электронный ресурс]. URL: https://docs.julialang.org/ (дата обращения: 11.09.2026).